

KabziMall

Teknik Temel & API Sözleşmesi

Veri Modeli · Fiyatlandırma Motoru · API · Akışlar — v1

Geliştirmeye geçiş köprüsü — bu doküman koda dönüştürülmek için yazıldı

28 Haziran 2026

İçindekiler

1. Amaç ve Kapsam

Bu doküman, KabziMall için hazırlanan PRD ve tıklanabilir prototiplerin üzerine, geliştiricinin doğrudan inşaya başlayabileceği teknik sözleşmeyi tanımlar: veri modeli, API yüzeyi, fiyatlandırma motorunun tek-kaynak formülleri ve kritik akışlar. Tasarım ve ürün kararları PRD'de; burada "nasıl kodlanır" sorusunun cevabı vardır.

Kapsam sınırı

Bu sürüm Faz 1 (v1) kapsamını hedefler: satışı başlatan Market + fiyat zekâsı çekirdeği. Şema ve API baştan çok-kiracılı (multi-tenant) kurgulanır; bugün tek işletme, yarın SaaS. Para birimi, ödeme sağlayıcısı gibi açık kararlar Bölüm 10'da listelenmiştir.

2. Mimari Özet

Katman	Teknoloji	Not
Mobil	React Native + TypeScript	TanStack Query, Zustand, MMKV, Firebase Push
Web	Next.js (App Router) + TS	SSR/SEO, ortak api-client
Admin/Intelligence	Next.js (admin app)	Market ile tip & UI paylaşır
Backend	NestJS + TypeScript	Modüler, REST; PostgreSQL + Redis
Veritabanı	PostgreSQL 15+	UUID, JSONB, partial index
Altyapı	Redis · Object storage · Queue	cache, job, görsel, bildirim/analiz işleri

2.1. Monorepo

```

/apps
  /mobile  React Native müşteri uygulaması
  /web     Next.js müşteri sitesi
  /admin   Next.js yönetim paneli + Intelligence
  /api     NestJS backend
/packages
  /types   paylaşılan TypeScript tipleri (DTO, enum)
  /pricing FİYATLANDIRMA MOTORU — tek kaynak (Bölüm 4)
  /api-client tip-güvenli API istemcisi
  /ui      ortak tasarım bileşenleri & token'lar
  /i18n    TR/EN çeviri kaynakları
  /config  ortak lint/tsconfig

```

Kritik kural: Fiyat ve marj hesaplamaları yalnızca packages/pricing içinde tanımlanır. Mobil, web, admin ve backend hepsi aynı fonksiyonu çağırır. Aynı formül iki yere kopyalanmaz.

3. Veri Modeli (PostgreSQL)

3.1. Konvansiyonlar

- **Birincil anahtar:** uuid (gen_random_uuid()). Tüm tablolarda id.
- **Çok-kiracılık:** tenant_id uuid (işletme kimliği) tüm iş tablolarında; satır seviyesi izolasyon (RLS) ile.
- **Zaman:** created_at, updated_at timestamptz default now(); yumuşak silme için deleted_at nullable.
- **Para:** Tutarlar integer kuruluş (minor units) olarak saklanır (ör. 3490 = 34,90 ₺) — float yuvarlama hatalarını önler. Sunum katmanı biçimlendirir.
- **Çok dilli:** Kısa alanlar *_tr / *_en kolonları; uzun/çok dilli içerik product_localizations tablosunda.

3.2. Market — ana tablolar

products — katalog çekirdeği:

Kolon	Tip	Açıklama
id	uuid	PK
tenant_id	uuid	işletme
category_id	uuid	FK categories
slug	text	URL/SEO, tenant içinde benzersiz
name_tr / name_en	text	ürün adı
sale_type	enum	WEIGHT · PIECE · BUNCH · PACK · VARIABLE_WEIGHT_PACK
unit_label_tr/en	text	kg · adet · demet · paket
base_price	int	kuruş, temel birim fiyatı (mağaza fiyatı)
discounted_price	int?	indirimli fiyat
origin_region	text?	menşei / yöre
is_active / is_featured	bool	yayın & vitrin
is_fresh_daily / is_local	bool	etiketler (günlük taze / yöresel)
min_order_qty / max_order_qty	numeric?	sipariş sınırı

Diğer market tabloları (özet): users, user_addresses, user_favorites, user_notification_preferences · categories, product_variants (varyant + fiyat çarpanı), product_images, product_tags, product_localizations · inventory_items, inventory_movements · carts, cart_items (qty, unit_price, note, substitution alanları) · orders, order_items, order_status_history, order_notes, order_refunds · delivery_zones, delivery_slots, delivery_assignments · coupons, coupon_redemptions, campaigns, campaign_products · banners, pages, faq_items · support_tickets, support_messages.

orders — sipariş başlığı (tartılı ürün ödeme nüansı dâhil):

Kolon	Tip	Açıklama
id / tenant_id / user_id	uuid	kimlikler
address_id / delivery_slot_id	uuid	teslimat
status	enum	Bkz. 6.1 sipariş yaşam döngüsü

Kolon	Tip	Açıklama
subtotal / discount_total	int	kuruş
delivery_fee / grand_total	int	kuruş
estimated_total	int	ön provizyon tutarı (tartılı için)
final_total	int?	paketleme sonrası kesinleşen tutar
payment_method / payment_status	enum	CARD·COD / PENDING·AUTHORIZED·CAPTURED·REFUNDED
substitution_preference	enum	REPLACE · CALL · REMOVE
customer_note / internal_note	text?	notlar
placed_at	timestamp tz	sipariş zamanı

order_items — kalem (sipariş edilen vs. paketlenen ayrımı kritik):

Kolon	Tip	Açıklama
id / order_id / product_id	uuid	kimlikler
variant_label	text	500 g · 1 kg · 1 adet
unit_price	int	kuruş (sipariş anındaki)
ordered_qty	numeric	istenen miktar
picked_qty	numeric?	paketlemede gerçekleşen (tartı)
line_total	int?	$\text{picked_qty} \times \text{unit_price}$
note	text?	ürün notu
substitution_status	enum?	NONE · SUBSTITUTED · REMOVED

3.3. Intelligence — fiyat zekâsı tabloları

Projenin farklılaştırıcı çekirdeği. Hepsinde tenant_id bulunur.

Tablo	Önemli kolonlar
hal_price_entries	product_id, price(int kuruş), unit, time_slot(enum: S08/S12/S16), source, captured_at, captured_by
competitor_groups	name (Premium/Orta/İndirim/Hızlı/Lokal), sort_order
competitors	name, group_id(FK), type, is_active
competitor_price_entries	product_id, competitor_id, price(int), source, captured_at, captured_by
cost_components	scope(enum: GLOBAL/CATEGORY/PRODUCT), ref_id, fire_rate(numeric %), packaging(int), labor(int), fuel(int), cold_storage(int), amortization(int), commission_rate(numeric %), tax_rate(numeric %)
pricing_rules	scope, ref_id, strategy(enum), params(jsonb), target_net_margin(numeric), floor_net_margin(numeric), psychological_round(bool)
price_history	product_id, old_price(int), new_price(int), strategy_applied, reason, net_margin(numeric), changed_by, changed_at
sales_snapshots	product_id, date, qty_sold(numeric), avg_price(int) — fiyat esnekliği/analiz için
margin_calcs	product_id, date, hal_avg(int), true_cost(int), suggested_price(int), net_margin(numeric) — rapor/denormalize

Tasarım notu: hal_price_entries ve competitor_price_entries append-only (her giriş yeni satır, zaman damgalı). Günlük ortalama ve trend bu satırlardan hesaplanır; geçmiş asla üzerine yazılmaz. price_history sayesinde "bu fiyat neden böyle?" her zaman izlenebilir.

4. Fiyatlandırma Motoru — Tek Kaynak Spesifikasyonu

packages/pricing içinde yaşar; saf (pure) fonksiyonlardır ve birim testlerle korunur. Tüm para girdileri kuruş (int) cinsindendir.

4.1. Çekirdek formüller

```
// fire (zayıt) maliyete TOPLANMAZ, BÖLÜNÜR.
fireCost = halAvg / (1 - fireRate) // fireRate: 0..1
directCost = fireCost + labor + packaging + fuel + coldStorage + amortization

// net marj (fiyat üzerinden), komisyon satış fiyatından düşülür:
netMargin(S) = (S - directCost - S * commissionRate) / S

// hedef marjı garanti eden fiyat (komisyon dâhil çözülmüş):
priceForMargin(m) = directCost / (1 - m - commissionRate)

// psikolojik yuvarlama (.90):
psych(x) = round(x) - 0.10
```

Neden bölme? (en kritik kural)

20,00 ₺/kg alışı · %20 fire → satılabilir = 0,80 kg → maliyet = 20 / 0,80 = 25,00 ₺/kg.
Yanlış (toplama) yöntem 24,00 ₺ derdi. %20 fire, maliyeti %20 değil %25 artırır.
Fire %30 olan maydanozda fark daha da büyük: 1 / 0,70 = +%43.

4.2. Strateji motoru

```
type Strategy =
| 'MARGIN' // priceForMargin(target) (önerilen varsayılan)
| 'COMP_AVG' // rakip ortalaması
| 'COMP_AVG_MINUS' // rakip ortalaması * (1 - x)
| 'MEDIAN' // rakip medyanı
| 'LOWEST' // min(rakip)
| 'GROUP_AVG' // seçili grup ortalaması (örn. Premium)
| 'FLOOR' // priceForMargin(floor)
| 'MANUAL'; // elle girilen

function suggestPrice(p, strategy, params): { price, netMargin, floored } {
  let raw = strategyValue(p, strategy, params);
  let price = psych(raw);
  let floored = false;
  if (netMargin(price) < p.floorNetMargin) { // taban marj koruması
    price = psych(priceForMargin(p.floorNetMargin));
    floored = true;
  }
  return { price, netMargin: netMargin(price), floored };
}
```

4.3. Doğrulama — birim test vakaları (Domates)

Girdi: hal_avg = 18,70 ₺ · fire %15 · işçilik 1,20 · ambalaj 0,70 · yakıt 0,50 · komisyon %3 · hedef marj %30 · taban %15.

Adım / strateji	Beklenen sonuç
fireCost = 18,70 / 0,85	22,00 ₺
directCost = 22,00 + 2,40	24,40 ₺
MARGIN: 24,40 / (1-0,30-0,03)	36,42 → psych 35,90 ₺ (net marj ≈ %29)

Adım / strateji	Beklenen sonuç
netMargin(35,90)	$\approx \%29,0$
COMP_AVG (49/46/42/44/39,9)	ort 44,18 → psych 43,90 ₺
MEDIAN	44,00 → 43,90 ₺
LOWEST	39,90 ₺
competitionIndex = $35,90 / 44,18 \times 100$	≈ 81 (rakipten ucuz)

KDV notu: KDV genelde satış fiyatına dâhildir ve devlete aittir; kârın parçası değildir. Net marj hesabında KDV ayrı tutulur (komisyon ve fire'dan farklı olarak fiyatı belirlemede kullanılmaz, sadece muhasebe/fatura tarafında işlenir).

5. API Sözleşmesi (REST)

5.1. Konvansiyonlar

- Taban yol: /api/v1. JSON gövde. Kimlik: Authorization: Bearer <jwt>.
- Çok-kiracılık: tenant token'dan çözülür; istemci tenant_id göndermez.
- **Hata biçimi:** { error: { code, message, details? } } + uygun HTTP kodu.
- Listeleme: ?page=&limit=&sort= ; cevap { data: [...], meta: { page, limit, total } }.
- Para alanları kuruş (int); istemci biçimlendirir.

5.2. Auth & Müşteri

```

POST /auth/otp/request    { phone }          → { challengeId }
POST /auth/otp/verify    { challengeId, code } → { accessToken, refreshToken, isNew }
POST /auth/profile       { firstName, lastName, email? }
POST /auth/refresh       { refreshToken }      → { accessToken }

GET /me                  → kullanıcı profili
GET /me/addresses        · POST /me/addresses · PATCH/DELETE /me/addresses/:id
POST /me/addresses/validate { lat,lng | il,ilce } → { inZone, zoneId }
GET /me/favorites        · POST/DELETE /me/favorites/:productId
GET /me/coupons

```

5.3. Katalog

```

GET /categories          → kategori ağacı
GET /products?category=&search=&tag=&sort=&page= → ürün listesi
GET /products/:slug      → ürün detay (+varyant, görsel, benzerler)
GET /products/featured | /fresh-today | /best-sellers
GET /baskets              (hazır sepetler) · GET /baskets/:id

```

5.4. Sepet & Sipariş

```

GET /cart                → aktif sepet + toplamlar
POST /cart/items         { productId, variant, qty, note? }
PATCH /cart/items/:id   { qty } | DELETE
POST /cart/coupon       { code } · DELETE /cart/coupon
PATCH /cart/substitution { preference: REPLACE|CALL|REMOVE }

POST /checkout/slots    { addressId } → uygun teslimat slotları
POST /orders            { addressId, slotId, paymentMethod, note? }
→ sipariş oluşur, kart ise estimated_total kadar PRE-AUTH alınır
GET /orders · GET /orders/:id · GET /orders/:id/status
POST /orders/:id/reorder → ürünleri sepete kopyalar
POST /orders/:id/issues { type, description } (eksik/yanlış ürün)

```

5.5. Intelligence (yetki: fiyat yöneticisi+)

```

// Hal fiyatları
GET /intel/hal?date=          → ürün × slot ızgarası + günlük ort.
POST /intel/hal/entries      { productId, slot, price, source } (append-only)
POST /intel/hal/bulk         { date, entries:[...] } (Saha Modu toplu kayıt)

// Rakipler
GET /intel/competitors · /intel/competitor-groups
GET /intel/competitor-prices?productId=&date= → değerler + min/max/avg/median
POST /intel/competitor-prices/entries { productId, competitorId, price }

// Maliyet & öneri

```

```

GET /intel/cost/:productId → directCost + kırılım + senaryo
PUT /intel/cost-components { scope, refId, ...fields }
POST /intel/price/suggest { productId, strategy, params } → { price, netMargin, floored }
POST /intel/price/apply { productId, price, strategy } → products.base_price güncelle + price_history
POST /intel/price/bulk-apply{ filter, strategy, params } → toplu güncelle (önizleme döner)

// Analiz
GET /intel/dashboard?date= → KPI'lar, trend, riskli ürünler
GET /intel/analytics/weekly | /monthly | /elasticity

```

Örnek — öneri isteği/yanıtı:

```

POST /api/v1/intel/price/suggest
{ "productId": "...", "strategy": "MARGIN", "params": { "targetMargin": 0.30 } }

200 OK
{ "price": 3590, "netMargin": 0.290, "competitionIndex": 81,
  "directCost": 2440, "floored": false, "currency": "TRY-minor" }

```

5.6. Operasyon (admin)

```

Ürün/kategori/stok CRUD · /admin/products /admin/categories /admin/inventory
GET /admin/orders?status=&slot=&q= · GET /admin/orders/:id
PATCH/admin/orders/:id/status
// Toplama/paketleme:
GET /admin/picking/queue
POST /admin/orders/:id/pick { items:[{ itemId, pickedQty, substitution? }] }
POST /admin/orders/:id/pack → final_total hesaplanır, kart PRE-AUTH → CAPTURE
POST /admin/orders/:id/deliver{ courierId, status }
Kampanya/kupon/banner CRUD · /admin/campaigns /admin/coupons /admin/banners

```

6. Kritik Akışlar

6.1. Sipariş yaşam döngüsü (status enum)

PENDING_PAYMENT → PAYMENT_CONFIRMED → CONFIRMED → PICKING → PACKING
→ READY_FOR_DELIVERY → OUT_FOR_DELIVERY → DELIVERED
yan durumlar: CANCELLED · PARTIALLY_FULFILLED · REFUNDED

6.2. Tartılı ürün ödeme (pre-auth → capture)

1. Müşteri siparişi onaylar; kart ise estimated_total kadar provizyon (AUTHORIZED) alınır.
2. Paketlemede personel gerçek gramajı (picked_qty) girer; sistem final_total hesaplar.
3. final_total ≤ estimated_total ise o tutar çekilir (CAPTURED); fazlaysa kurallı üst sınır/onay akışı uygulanır.
4. Fark müşteriye bildirilir; iade gerekiyorsa REFUNDED. Sözleşme ve sepet ekranında bu durum baştan açıklanır.

6.3. Fiyat öner → uygula döngüsü

5. Hal & rakip fiyatları girilir (append-only).
6. cost-components + günlük hal ort. → directCost hesaplanır.
7. /price/suggest seçilen stratejiyle fiyatı üretir (taban marj korumalı).
8. /price/apply ürünün base_price'ını günceller ve price_history'e yazar; Market anında bu fiyatı yayınlar.

7. Roller ve Yetki Matrisi

Rol	Erişim
Süper Admin	Her şey + kullanıcı/rol + ayarlar
Operasyon yöneticisi	Ürün, stok, sipariş, kampanya; Intelligence okuma
Fiyat yöneticisi	Intelligence tam (hal/rakip/maliyet/öneri/apply)
Paketleme personeli	Picking/packing endpoint'leri; fiyat yok
Kurye	Teslimat durumu güncelleme
Müşteri destek	Sipariş okuma, destek talepleri, iade talebi
Müşteri	Kendi sepeti/siparişi/profil

Yetkilendirme backend'de policy/guard ile; satır seviyesinde tenant_id (RLS) zorunlu.

8. Önerilen Build Sırası & Definition of Done

9. Altyapı: monorepo, NestJS iskelet, PostgreSQL şema migrasyonları, auth (OTP).
10. packages/pricing: formüller + birim testler (Bölüm 4.3 vakaları yeşil olmadan ilerleme).
11. Katalog API + Intelligence hal/rakip/maliyet/öneri endpoint'leri.
12. Sepet + checkout + sipariş (pre-auth/capture); paketleme akışı.
13. Mobil & web müşteri ekranları (prototiplerden); admin Intelligence ekranları.
14. Analiz/dashboard + price_history; KVKK/sözleşme; pilot bölge lansmanı.

Her endpoint için "done": tip tanımı (packages/types), girdi doğrulama, yetki guard'ı, hata biçimi, en az bir entegrasyon testi, ve para alanlarının kuruş cinsinden tutarlılığı.

9. Para, Yuvarlama ve Birim Notları

- Saklama: integer kuruş. Görüntüleme: tr-TR biçimi (34,90 ₺).
- Tartılı ürün: fiyat birim başına; $tutar = unit_price \times picked_qty$ (gram → kg ondalık).
- Psikolojik yuvarlama yalnızca öneri üretiminde; mağaza fiyatı net tutulur.
- KDV fatura/muhasebe katmanında; net marj hesabında ayrı.

10. Açık Kararlar (netleştirilecek)

Konu	Karar gereği
Ödeme sağlayıcı	iyzico / PayTR vb. — pre-auth + partial capture desteği teyit edilmeli
Para temsili	Öneri: integer kuruş (onay)
Otomatik rakip toplama	MVP'de elle; otomasyon hukuki görüş gerektirir (PRD 3.3)
Kurye uygulaması	v1 panelden; v2 ayrı uygulama
E-fatura entegrasyonu	sağlayıcı seçimi & fatura akışı
Çok-kiracılık zamanlaması	baştan tenant_id (önerilir) vs. sonradan migrasyon

Sonuç: Bu sözleşme, packages/pricing testleri yeşile döndüğü an geliştirmenin güvenle başlayabileceği bir temel sağlar. Fiyat motoru tek kaynak olduğu sürece mobil, web ve panel her zaman aynı sayıyı gösterir.